ELP USB Camera

Multi-channel Global Synchronization Linux SDK

Development Manual

2026-6-25 Ver.A

ELP

www.elpcctv.com

Version Release Notes

Version No.	Revision Date	Revision Description
A	2026-6-25	Initial Draft Created

Catalogue

VERSION RELEASE NOTES **2**

1. Overview **4**

1.1 Purpose 4
1.2 Introduction to ELP SDK 4-6

2. Structure Description **6**

2.1 Image Encoding Type 6
2.2 Device Information 6
2.3 Image Format Information 7
2.4 Parsed Image Information 7
2.5 Multi-channel Synchronous Image Group 7
2.6 IMU and Magnetometer Information 7-8

3. API Function Description **8**

3.1 SYNC_Initialize 8
3.2 SYNC_Release 8
3.3 SYNC_GetVersion 9
3.4 SYNC_GetLastError 9
3.5 SYNC_GetErrorText 9
3.6 SYNC_EnumDevices 9-10
3.7 SYNC_OpenDevice 10
3.8 SYNC_CloseDevice 10
3.9 SYNC_GetDeviceFormats 10-11
3.10 SYNC_SetDeviceFormat 11
3.11 SYNC_GetDeviceFormatIndex 11
3.12 SYNC_SetImageCallback 11-12
3.13 SYNC_SetCaptureFormat 12
3.14 SYNC_StartCapture 12
3.15 SYNC_StartCaptureByIndices 12-13
3.16 SYNC_StopCapture 13
3.17 SYNC_IsCapturing 13
3.18 SYNC_ SaveNV12ToJPG 13-14
3.19 SYNC_ GetFrameRate 14
3.20 Callback Function Pointer 14

1. Overview

1.1 Purpose

This document describes the API usage of ELP Multi-camera Synchronous Linux SDK (hereinafter referred to as SDK).

Developers can use this SDK under Linux-Ubuntu systems to develop software for synchronously acquiring images from multiple USB cameras. There is no need to concern about underlying implementations. By calling a small set of APIs, developers can obtain exposure timestamps, timestamped IMU data and image data (NV12 format) from multiple devices. Frame synchronization can be performed among multiple cameras, and image data and IMU data of a single device can also be synchronized. The synchronized image data can be directly used for image processing with OpenCV or video encoding.

This SDK is only applicable to ELP global shutter USB3.0 cameras with timestamp function (solutions such as SC133HGS, AR0234, OS05B10).

1.2 Introduction to ELP SDK

The SDK supports Ubuntu 20.04 and later systems.

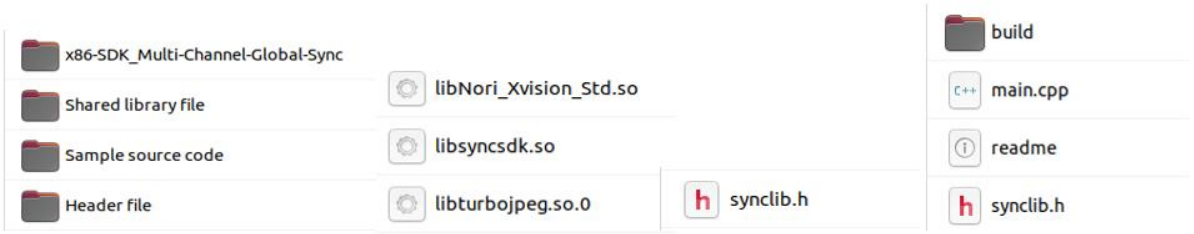
The SDK runs on three platforms: X86, RK35 series, and NVIDIA Jetson series (Xavier and Orin).

Each platform requires its corresponding library files, while the API usage and sample code remain identical.

The SDK is developed in C/C++ and can also be invoked via Python.

SDK contents include dynamic library files, header files and sample source code.

Take the X86 platform as an example：



In the sample source code, the build directory stores the shared library files, and main.cpp contains the sample code.

Compilation Command:：g++ main.cpp -o test -I./ -L./build -lsyncsdk -lturbojpeg -lpthread

Compile to generate the executable file test

Command to run the program： LD_LIBRARY_PATH=./build ./test

After launching the program, it continuously prints parameters of each camera, exposure timestamps and IMU data across the entire terminal screen in a loop (refer to the figure below).

Press the **Spacebar** to take photos. A synchronously-stitched multi-camera frame will be saved in JPEG format. Meanwhile, the exposure timestamp and IMU data corresponding to this frame will be saved in text format.

For detailed SDK usage and synchronization principles, please refer to the comments in readme and main.cpp.

```
synccap SDK Testing Program
SDK Version: MCCS Sync NV12 SDK v2.2.1

Press Space to capture...

[Device Enumeration] Detected 4 devices:
  Device 0: Camera101: Camera101, vidpid=32e4a101, useEndTime=0
  Device 1: Camera102: Camera102, vidpid=32e4a102, useEndTime=0
  Device 2: Camera103: Camera103, vidpid=32e4a103, useEndTime=0
  Device 3: Camera104: Camera104, vidpid=32e4a104, useEndTime=0
[Format] Device 0 supports 2 types:
    0: MJPG 2080x1200 60fps
    1: MJPG 2080x1200 30fps
[Format Setup] Device 0 (pid=0) set to format: 2080x1200 60fps
[Format] Device 1 supports 2 types:
    0: MJPG 2080x1200 60fps
    1: MJPG 2080x1200 30fps
[Format Setup] Device 1 (pid=1) set to format: 2080x1200 60fps
[Format] Device 2 supports 2 types:
    0: MJPG 2080x1200 60fps
    1: MJPG 2080x1200 30fps
[Format Setup] Device 2 (pid=2) set to format: 2080x1200 60fps
[Format] Device 3 supports 2 types:
    0: MJPG 2080x1200 60fps
    1: MJPG 2080x1200 30fps
[Format Setup] Device 3 (pid=3) set to format: 2080x1200 60fps

[Device Auto-Selection] Automatically selected all 4 devices:
  Device 0: Camera101: Camera101, pid=0
  Device 1: Camera102: Camera102, pid=1
  Device 2: Camera103: Camera103, pid=2
  Device 3: Camera104: Camera104, pid=3
```

```
Camera 0: NV12 1920x1200, size=3456000 bytes, fps=61.02
exposure: start=   142261060 us end=   142266038 us, duration=    4978 us
[g] 0x0 Acc=(    2.807617,  979.492188,  207.031250) Gyro=(   -0.488281,   -0.274658,    0.793457) time=  142249406 us
[g] 0x1 Acc=(    2.685547,  980.102539,  207.641602) Gyro=(   -0.457764,   -0.274658,    0.823975) time=  142251070 us
[g] 0x2 Acc=(    3.295898,  980.346680,  208.618164) Gyro=(   -0.457764,   -0.274658,    0.793457) time=  142252734 us
[g] 0x3 Acc=(    3.295898,  979.736328,  208.374023) Gyro=(   -0.427246,   -0.274658,    0.793457) time=  142254398 us
[g] 0x4 Acc=(    4.028320,  979.125977,  208.007812) Gyro=(   -0.427246,   -0.305176,    0.793457) time=  142256062 us
[g] 0x5 Acc=(    5.493164,  979.370117,  209.594727) Gyro=(   -0.427246,   -0.274658,    0.793457) time=  142257726 us
[g] 0x6 Acc=(    5.126953,  978.881836,  210.571289) Gyro=(   -0.396729,   -0.244141,    0.793457) time=  142259390 us
[g] 0x7 Acc=(    3.906250,  977.783203,  209.716797) Gyro=(   -0.335693,   -0.244141,    0.793457) time=  142261054 us
[g] 0x8 Acc=(    4.272461,  978.271484,  209.228516) Gyro=(   -0.366211,   -0.274658,    0.762939) time=  142262704 us
[g] 0x9 Acc=(    4.882812,  979.492188,  209.838867) Gyro=(   -0.366211,   -0.244141,    0.762939) time=  142264368 us

Camera 1: NV12 1920x1200, size=3456000 bytes, fps=61.33
exposure: start=   142261059 us end=   142266037 us, duration=    4978 us
[g] 0x0 Acc=(  -42.968750,  979.003906,  202.392578) Gyro=(   -0.152588,   -0.091553,   -0.274658) time=  142249405 us
[g] 0x1 Acc=(  -42.968750,  978.515625,  203.369141) Gyro=(   -0.122070,   -0.091553,   -0.244141) time=  142251068 us
[g] 0x2 Acc=(  -43.579102,  979.736328,  203.979492) Gyro=(   -0.061035,   -0.091553,   -0.213623) time=  142252733 us
[g] 0x3 Acc=(  -42.968750,  981.323242,  204.101562) Gyro=(   -0.030518,   -0.122070,   -0.183105) time=  142254397 us
[g] 0x4 Acc=(  -41.503906,  983.154297,  203.979492) Gyro=(    0.000000,   -0.091553,   -0.152588) time=  142256060 us
[g] 0x5 Acc=(  -41.748047,  983.520508,  204.467773) Gyro=(    0.000000,   -0.061035,   -0.183105) time=  142257725 us
[g] 0x6 Acc=(  -42.724609,  980.834961,  205.444336) Gyro=(   -0.030518,   -0.030518,   -0.213623) time=  142259388 us
[g] 0x7 Acc=(  -42.358398,  977.783203,  206.298828) Gyro=(   -0.061035,   -0.061035,   -0.213623) time=  142261052 us
[g] 0x8 Acc=(  -42.236328,  977.416992,  206.542969) Gyro=(   -0.091553,   -0.122070,   -0.213623) time=  142262703 us
[g] 0x9 Acc=(  -42.602539,  979.614258,  206.176758) Gyro=(   -0.030518,   -0.122070,   -0.244141) time=  142264367 us

Camera 2: NV12 1920x1200, size=3456000 bytes, fps=61.45
exposure: start=   142261059 us end=   142266037 us, duration=    4978 us
[g] 0x0 Acc=( -156.250000,  983.154297,  -56.762695) Gyro=(    1.007080,    0.305176,    0.335693) time=  142249404 us
[g] 0x1 Acc=( -156.860352,  984.375000,  -56.396484) Gyro=(    1.007080,    0.305176,    0.305176) time=  142251068 us
[g] 0x2 Acc=( -156.494141,  984.252930,  -56.762695) Gyro=(    0.976562,    0.274658,    0.305176) time=  142252732 us
[g] 0x3 Acc=( -155.151367,  983.520508,  -57.128906) Gyro=(    1.007080,    0.274658,    0.366211) time=  142254396 us
[g] 0x4 Acc=( -153.808594,  983.642578,  -56.762695) Gyro=(    1.037598,    0.274658,    0.335693) time=  142256060 us
[g] 0x5 Acc=( -153.808594,  983.154297,  -56.152344) Gyro=(    1.068115,    0.274658,    0.335693) time=  142257724 us
[g] 0x6 Acc=( -153.564453,  982.666016,  -55.786133) Gyro=(    1.068115,    0.274658,    0.335693) time=  142259388 us
[g] 0x7 Acc=( -152.832031,  983.520508,  -55.541992) Gyro=(    1.037598,    0.244141,    0.335693) time=  142261052 us
[g] 0x8 Acc=( -153.076172,  983.520508,  -55.053711) Gyro=(    1.068115,    0.244141,    0.335693) time=  142262703 us
[g] 0x9 Acc=( -152.709961,  982.666016,  -54.199219) Gyro=(    1.098633,    0.274658,    0.335693) time=  142264367 us

Camera 3: NV12 1920x1200, size=3456000 bytes, fps=61.20
exposure: start=   142261059 us end=   142266037 us, duration=    4978 us
[g] 0x0 Acc=(   -6.713867,  958.496094,  274.169922) Gyro=(    0.000000,   -0.122070,    0.000000) time=  142249405 us
[g] 0x1 Acc=(   -7.080078,  957.641602,  275.878906) Gyro=(   -0.030518,   -0.091553,    0.000000) time=  142251069 us
[g] 0x2 Acc=(   -6.713867,  958.618164,  275.878906) Gyro=(    0.000000,   -0.061035,    0.000000) time=  142252732 us
[g] 0x3 Acc=(   -6.347656,  959.960938,  275.512695) Gyro=(    0.061035,   -0.061035,    0.000000) time=  142254396 us
[g] 0x4 Acc=(   -5.737305,  960.693359,  275.268555) Gyro=(    0.061035,   -0.091553,    0.000000) time=  142256061 us
[g] 0x5 Acc=(   -5.981445,  961.303711,  274.658203) Gyro=(    0.030518,   -0.152588,    0.000000) time=  142257725 us
[g] 0x6 Acc=(   -6.103516,  960.327148,  274.536133) Gyro=(    0.030518,   -0.183105,    0.030518) time=  142259389 us
[g] 0x7 Acc=(   -5.859375,  958.374023,  274.780273) Gyro=(    0.000000,   -0.152588,    0.061035) time=  142261053 us
[g] 0x8 Acc=(   -6.103516,  957.641602,  274.658203) Gyro=(    0.000000,   -0.091553,    0.030518) time=  142262703 us
[g] 0x9 Acc=(   -6.225586,  958.862305,  275.024414) Gyro=(    0.030518,   -0.091553,   -0.030518) time=  142264367 us
```

2. Structure Descriptions

2.1 Image Encoding Type

```
typedef enum SyncCamFormat {
    SYNC_FORMAT_UNKNOWN = 0, /* Unknown or unsupported image format. */
    SYNC_FORMAT_MJPG = 1, /* MJPG compression format. */
    SYNC_FORMAT_YUY2 = 2, /* YUV422 packed format. */
    SYNC_FORMAT_NV12 = 3    /* NV12 format: Y-plane + interleaved UV-plane. */
} SyncCamFormat;
```

2.2 Device Information

```
typedef struct SyncDeviceInfo {
    char name[256]; /* Device name, e.g. Camera0001: Camera0001. */
    char vidpid[16]; /* Device VID/PID string, e.g. 32e4b001. */
    char path[256];    /* Device path, e.g. /dev/video0. */
    uint32_t number; /* Logical device ID, e.g. Camera0001 => 0, Camera0002 => 1. */
```

```
    bool isExist;    /* Whether the device is currently present. */
    bool isOpened; /* Whether the device is currently opened. */
} SyncDeviceInfo;
```

2.3 Image Format Information

```
  typedef struct SyncFormatInfo {
    SyncCamFormat fmt; /* Image format type. */
    int width; /* Image width. */
    int height; /* Image height. */
    int fps; /* Image frame rate. */
} SyncFormatInfo;
```

2.4 Parsed Image Information

```
   typedef struct SyncCamData {
    const uint8_t* data; /* Start address of NV12 pixel data. */
    int width; /* Image width. */
    int height; /* Image height.*/
    int bufSize; /* Total byte size of the image buffer. */
    SyncCamFormat format; /* Current image format, normally SYNC_FORMAT_NV12. */
    uint64_t startExpouseTime; /* Exposure start timestamp. */
    uint64_t endExpouseTime; /* Exposure end timestamp. */
    SyncICM42688Data imu_data[SYNC_IMU_SAMPLE_COUNT]; /* IMU data array attached to current
frame. */
    SyncAK09940Data mtt_data[SYNC_MTT_SAMPLE_COUNT]; /* Magnetometer data array attached to
current frame. */
    int deviceIdx; /* SDK logical device ID: Camera0001 => 0, Camera0002 => 1 ... */
    uint32_t rawIndex; /* Raw device index from underlying capture library. */
    bool isValid; /* Whether current frame is valid. */
} SyncCamData;
```

2.5 Multi-channel Synchronized Image Group

```
typedef struct SyncCamGroupData {
    SyncCamData camData[SYNC_MAX_CAMERA_COUNT]; /* Image data of each camera within current
group. */
    int deviceCount; /* Actual number of devices contained in current group. */
} SyncCamGroupData;
```

2.6 IMU and Magnetometer Information

```
typedef struct SyncICM42688Data {
```

```
    uint64_t uTime; /*Sampling timestamp. */
    float fAccData_X; /*X-axis acceleration. */
    float fAccData_Y; /* Y-axis acceleration. */
    float fAccData_Z; /* Z-axis acceleration. */
    float fGyroData_X; /* X-axis gyroscope angular velocity. */
    float fGyroData_Y; /* Y-axis gyroscope angular velocity. */
    float fGyroData_Z; /* Z-axis gyroscope angular velocity. */
} SyncICM42688Data;

typedef struct SyncAK09940Data {
    uint64_t uTime; /* Sampling timestamp. */
    int32_t iX; /* X-axis raw magnetic field data. */
    int32_t iY; /* Y-axis raw magnetic field data. */
    int32_t iZ; /*Z-axis raw magnetic field data. */
    float Temp; /* Sensor temperature. */
    int32_t iStatusBit; /*Status bit. */
} SyncAK09940Data;
```

3. API Function Descriptions

3.1 SYNC_Initialize

Function Prototype: int SYNC_Initialize();
Function: Initialize SDK resources Parameters: None Return Value: Returns 1 on success, 0 on failure Usage Note: This function must be called to perform initialization before invoking any other APIs.

3.2 SYNC_Release

Function Prototype: void SYNC_Release();
Function: Release SDK resources Parameters: None Return Value: None

Usage Note: Call this function upon program exit.

3.3 SYNC_GetVersion

Function Prototype: const char* SYNC_GetVersion();

Function: Retrieve SDK version information

Parameters: None

Return Value: Returns a string containing version information

Usage Note: None

3.4 SYNC_GetLastError

Function Prototype: const int SYNC_GetLastError();

Function: Obtain error code

Parameters: None

Return Value: Returns 0 if no error occurs; returns an integer error code if an error occurs

Usage Note: None

3.5 SYNC_GetErrorText

Function Prototype: const char* SYNC_GetErrorText(int errorCode);

Function: Get detailed error information

Parameters: errorCode: Integer-type error code

Return Value: Returns the string message corresponding to the error code

Usage Note: None

3.6 SYNC_EnumDevices

Function Prototype: int SYNC_EnumDevices(SyncDeviceInfo* devices, int maxCount, int* count)

Function: Enumerate device information

Parameters: devices：Array for storing device information

maxCount：Maximum number of devices to enumerate

count：Returns the actual number of detected devices

Return Value: Returns 1 on successful enumeration, 0 on enumeration failure

Usage Note: None

3.7 SYNC_OpenDevice

Function Prototype: int SYNC_OpenDevice(uint32_t deviceIndex);

Function: Open a device

Parameters: deviceIndex: Index of the device to open (0, 1, 2, 3)

Return Value: Returns 1 on success, 0 on failure

Usage Note: None

3.8 SYNC_CloseDevice

Function Prototype: SYNC_CloseDevice(uint32_t deviceIndex);

Function: Close the specified device

Parameters: deviceIndex: Index of the device to close (0, 1, 2, 3)

Return Value: Returns 1 on success, 0 on failure

Usage Note: None

3.9 SYNC_GetDeviceFormats

Function Prototype: int SYNC_GetDeviceFormats(uint32_t deviceIndex, SyncFormatInfo* formats, int maxCount, int* count);

Function: Get supported formats of the device

Parameters:

deviceIndex：Device index (0, 1, 2, 3)

formats：Pointer of type SyncFormatInfo to store format information

maxCount：Maximum number of formats to obtain; pass the array size if formats points to an array

count：Integer pointer to hold the number of retrieved formats

Return Value: Returns 1 on success, 0 on failure

Usage Note: None

3.10 SYNC_SetDeviceFormat

Function Prototype: int SYNC_SetDeviceFormat(uint32_t deviceIndex, int formatIndex);

Function: Set device format

Parameters:

deviceIndex：Device index (0, 1, 2, 3)

formatIndex：Index of the target format to set

Return Value: Returns 1 on success, 0 on failure

Usage Note: None

3.11 SYNC_GetDeviceFormatIndex

Function Prototype: int SYNC_GetDeviceFormatIndex(uint32_t deviceIndex);

Function: Get index of the currently active format

Parameters: deviceIndex: Device index (0, 1, 2, 3)

Return Value: Returns current format index on success, -1 on failure

Usage Note: None

3.12 SYNC_SetImageCallback

Function Prototype: int SYNC_SetImageCallback(ImageCallback callback, void* userData);

Function: Register synchronization callback function

Parameters:

callback：User-defined callback function

userData：User-supplied parameter

Return Value: Returns 1 on success, 0 on failure

Usage Note: The callback function is used to receive synchronized image data.

3.13 SYNC_SetCaptureFormat

Function Prototype: void SYNC_SetCaptureFormat(SyncCamFormat format);

Function: Set output image format

Parameters:

format：Target format to set

Currently only SYNC_FORMAT_NV12 is supported.

Return Value: None

Usage Note: None

3.14 SYNC_StartCapture

Function Prototype: int SYNC_StartCapture(int deviceCount);

Function: Start capture for the first deviceCount existing devices starting from index 0

Parameters: deviceCount: Number of devices to enable

Return Value: Returns 1 on success, 0 on failure

Usage Note: None

3.15 SYNC_StartCaptureByIndices

Function Prototype: int SYNC_StartCaptureByIndices(const int* deviceIndices, int deviceCount);

Function: Start capture for devices specified by logical device indices

Parameters:

deviceIndices：Array of logical device indices, e.g. {0, 2, 4}

deviceCount：Number of entries in the deviceIndices array

Return Value: Returns 1 on success, 0 on failure

Usage Note: None

3.16 SYNC_StopCapture

Function Prototype: int SYNC_StopCapture(void);

Function: Stop image capture

Parameters: None

Return Value: Returns 1 on success, 0 on failure

Usage Note: None

3.17 SYNC_IsCapturing

Function Prototype: bool SYNC_IsCapturing(void);

Function: Check whether capture is in progress

Parameters: None

Return Value: Returns true if capture is active; returns false otherwise

Usage Note: None

3.18 SYNC_ SaveNV12ToJPG

Function Prototype: bool SYNC_SaveNV12ToJPG(const uint8_t* nv12_buf, int w, int h, const char* filePath, int quality);

Function: Save NV12-format image as JPEG file

Parameters:

nv12_buf:NV12 image data buffer

w: Image width

h: Image height

filePath: Output path for the JPEG file

quality: JPEG compression quality, valid range: 1~100

Return Value: Returns true if saving succeeds; false if saving fails

Usage Note: None

3.19 SYNC_ GetFrameRate

Function Prototype: float SYNC_GetFrameRate(uint32_t deviceIndex);

Function: Get decoded frame rate

Parameters: deviceIndex: Logical device index

Return Value: Returns frame-rate value of the specified device

Usage Note: None

3.20 Callback Function Pointer

Function Prototype: typedef void (*ImageCallback)(const SyncCamGroupData* image, void* userData);

Function: User-defined callback function prototype

Parameters:

image： Pointer to a group of synchronized image data, which contains multiple SyncCamData entries

userData： Custom user parameter passed during callback registration, passed back unchanged

Return Value: None

Usage Note: None